

ROB498 Robotics Capstone Design

RoboRangers - Final Report

Rafiu Hossain
Justin Narayan
Yang Niu
Sujit Peramanu
Isabelle Tung



April 10, 2026

1 Design Overview

Wild pigs are one of the most destructive invasive species in North America, causing billions of dollars in agricultural damage while disrupting ecosystems and spreading disease [1–3]. Their rapid population growth, adaptability, and lack of natural predators have made them particularly difficult to control [1, 3]. Despite their impact, the primary challenge is not simply detecting these animals, but continuously tracking and managing them in large, unstructured environments. Wild pigs are fast, highly adaptive, and often active at night, making real-time monitoring both technically difficult and resource-intensive [3–6].

Current wildlife management approaches rely on public sighting reports, trail cameras, and helicopter-assisted operations [2, 3, 7]. While these methods provide useful information, they are expensive, labor-intensive, and difficult to scale across large geographic regions [2, 4]. Although unmanned aerial vehicles (UAVs) have been introduced to assist with detection, they are typically manually operated and limited to locating animals rather than actively tracking or interacting with them [6, 8, 9]. As a result, a critical gap exists: there is no scalable system that enables continuous, real-time tracking and intervention.

To address this gap, we developed Robo-Ranger, an autonomous drone system designed to detect, follow, and assist in the capture of invasive wild pigs. The system integrates onboard computer vision for target detection, autonomous control for real-time tracking, and a lightweight servo mechanism to deploy a net from above to support GPS tagging operations, while keeping a human operator in the loop for safety-critical decisions. In a typical deployment, a field operator launches the drone in an area of interest, then the system autonomously searches for a target, tracks its movement, and positions itself to support capture or GPS tagging. Once the drone is correctly positioned, the operator can manually trigger net deployment. By reducing the need for continuous manual piloting, the system enables monitoring over larger areas while improving operational efficiency and safety.

Over the course of the term, we designed and implemented a functional prototype that demonstrates the core components of this system. Real-time target (mock wild pig plushie) detection was successfully developed and validated independently, stable tracking behavior was achieved using motion capture-based target inputs, and a reliable net deployment mechanism was implemented and tested. While full integration of perception and control was limited by system-level constraints, particularly related to power and hardware reliability, the prototype demonstrates the feasibility of the overall system architecture and highlights key system-level challenges associated with real-world deployment.

Our design process was guided by three key principles: modularity, safety, and incremental validation. The system was decomposed into independent subsystems: computer vision, control, and net deployment, enabling parallel development and clear separation of functionality. Each subsystem was first developed and tested independently before being progressively integrated, allowing issues to be isolated and addressed early; however, some system-level constraints only became apparent during later stages of integration. Safety was prioritized throughout, with mechanisms such as geofencing and Vicon-based fallback positioning ensuring that new functionalities could be evaluated without risking catastrophic

failure. Our team organization reflected this approach, with members taking ownership of individual subsystems while collaborating during integration and validation phases, enabling efficient development while highlighting the importance of coordinated system testing and debugging.

2 Background

All utilized hardware and firmware systems – including the drone chassis, motors, propellers, Cube Orange+, Jetson Nano, PX4 modules, MAVROS communications architecture, RealSense T265, IMX219, and all required firmware drivers – were provided by the ROB498 Teaching Team; these were not key design elements. However, our team made several key design decisions in selecting our computer vision approach, software architecture, and mechatronics assembly structure.

2.1 Computer Vision

For onboard perception, we wanted to incorporate both the IMX219 and the RealSense T265 for end-to-end pose estimation. Although perception approaches exist that bypass full object pose recovery, we believe that, in spite of our hardware constraints, onboard full object detection is still feasible on Jetson Nano. A neural network approach was used instead of classical methods as it is far more robust to lighting, motion, and variation in object appearance. YOLOv8n was chosen due to the critical balance it offers between inference speed and detection accuracy on our edge device. Combined with the disparity map extracted depth projection, we can recover the full object pose in our frame of choice for downstream control processes.

2.2 State Machine

All drone control logic was implemented using a finite state machine architecture. Finite state machines are a frequent design pattern in robotics and electronics projects in both industry and hobbyist settings. Using state machines, the system proceeds through a set of discrete states, and at any point can transition to other available states from its current state based on external or internal stimuli (i.e. a user-sent command or an internal trigger like a timer). In implementing a state machine, the main loop of operations is broken into two stages: the state transition stage and the state process stage, as shown in Figure 1. At the start of each loop, all relevant stimuli are checked to identify if a state transition should occur – then, the appropriate processes depending on the current state are executed [10]

There are a number of key benefits to this design pattern. First, all behaviours are very traceable and predictable: the bisection of operational stages ensures that, for at least one loop, every state is executed, preventing erroneous designs that may prematurely transition past states without executing all relevant commands. Second, the modular design allows for easy modification: adding new functionality is as simple as providing the appropriate stimuli for transition and relevant processes at that state, with no changes required to operations

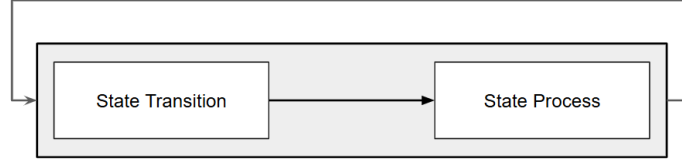


Figure 1: Abstracted state machine architecture operations loop.

anywhere else in the software stack. Third, state machines can easily prioritize safety with “early-exit” stimuli: performing state transition checks before state processes ensures that any high-priority stimuli such as abort commands or emergency-stop detections can be received and executed immediately, rather than being potentially buried in a deluge of other process logic in a less streamlined architecture.

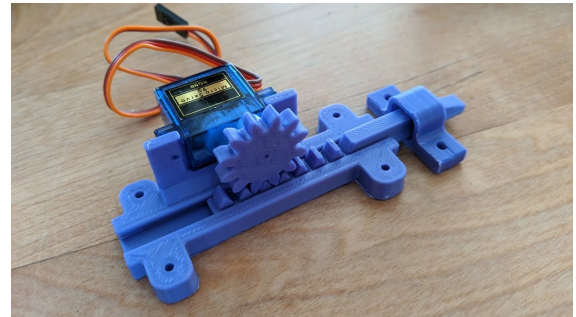
Due to our team’s familiarity with state machine approaches from industry and academic experiences, as well as their myriad benefits, we implemented this architecture.

2.3 Net Deployment

There are currently no standards for deploying a net from a drone. In reviewing hobbyist design projects (shown in Figure 2a), we identified some common functional elements: mechanisms to trigger deployment, a method to release or launch the net, and features that ensure the net spreads during descent rather than folding up.



(a) A sample net gun design from <https://www.netgun.com>.



(b) A bolt-latch CAD from <https://www.thingiverse.com/thing:4379943/files>.

Figure 2: Sample net deployment mechanisms.

Some reference designs used active launch mechanisms, such as compressed gas or spring-loaded systems, to project the net toward a target. However, these approaches introduced additional complexity and safety concerns when used on a drone. Instead, we drew inspiration from passively deployed systems, such as cast fishing nets, which are released and allowed to fall under gravity. In particular, cast nets often incorporate weights along their edges to improve spreading. For triggering, a variety of mechanisms are used, including latch-based systems such as the bolt-latch design shown in Figure 2b, which securely holds the net during flight and releases it when actuated.

3 Objectives

A successful real-world solution would survey vast wild pig ranges (in all environments, weather, and times of day) and safely capture wild pigs for manual catch-and-release tagging by wildlife experts. The design goals are:

- (a) **accurately identifying** wild pigs in relevant environments
- (b) **autonomously following** wild pigs’ movements from a fixed relative position
- (c) **deploying a net** from above to fully contain stationary wild pigs
- (d) **safely returning** to and launching from a designated “home” position, and
- (e) **endurance** for a sufficiently long flight-time to survey a “large” area

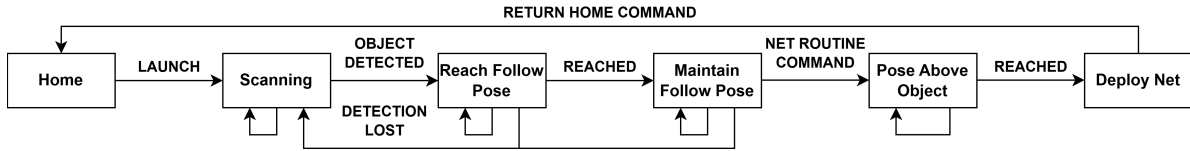


Figure 3: Proposed state machine architecture.

The developed prototype only serves as a proof-of-concept for the target detection, tracking, and capture functionalities of the proposed design for the real-world solution. Its scope is intentionally simplified to operate within a small, well-lit indoor environment (Myhal 580), and it performs object detection on a surrogate pig prototype (a plushie on an RC car, as in Figure 11) rather than a hyper-realistic wild pig representation. Due to constraints on available hardware, including batteries and processing units, the endurance objective was not evaluated, as it fell outside of the scope of the project given battery and power limitations.

For object detection, design objectives consist of (1) autonomously estimating the target’s position using onboard sensor data, and (2) determining the goal position of the drone based on this estimate. The target localization estimates produced should be of high accuracy ($\geq 95\%$ true positive rate).

For tracking, design objectives consist of (1) generating a control input for the drone based on the received goal position, and (2) converging to the desired position using feedback control. The drone position should demonstrate minimal steady-state error with respect to the target pose ($\geq 95\%$ score assessed against drift zones tests from Flight Exercise 2). The drone should also converge to the target position with minimal overshoot ($\leq 10\%$).

For net deployment, design objectives consist of (1) enabling remote actuation by an operator, (2) mechanically releasing the net from the drone, and (3) enclosing the target upon deployment. The deployment of the net should be able to fully enclose the target with high consistency ($\geq 95\%$ success rate across trials).

For the net deployment subsystem, the functional requirements can be constructed separately for the net and deployment systems. The net must (1) be sufficiently large so as to fully enclose the pig prototype, and (2) not promote aerodynamic drag sufficient to promote upward motion. The deployment system must (1) attach securely to the drone chassis, (2)

contain the net during flight such that it does not interfere with propellers, (3) deploy the net such that it is unfurled upon descent, and (4) require minimal mass and power resources.



Figure 4: Proposed prototype components: **critical functionality**, **supplemental functionality**, **stretch goals**.

Several critical parameters dictate the design’s feasibility and performance. CV processing (“inference”) time is a key consideration; the inference must be accurate, but also run sufficiently fast to enable responsive control, as rapidly moving targets may evade tracking with excessive inference latency. Control parameters for the drone’s dynamic flight envelope (velocity, acceleration, and payload characteristics) must also be precisely tuned; sharper, more aggressive trajectories may enable defter repositioning or better following of fast targets, but erratic movements risk blurring images for CV, inducing instability, or overburdening the drone’s electronics. Beyond the prototype, a final design must balance the drone’s tightly-coupled hardware specifications (mass, battery capacity, size): larger drones can carry bigger nets (for full-size wild pigs) and will be more stable in intensive weather, but will consume more battery power, reducing flight-time.

The three subsystems can be assessed independently to prove the success of the design’s object detection, tracking, and capturing capabilities, respectively. While full system integration was limited within the prototype, this revealed important system-level constraints, particularly related to overwhelmed communication networks or power usage, which are primarily hardware dependent, and would be addressed in a real-world implementation using more capable platforms.

The design goals did not change throughout the prototype’s implementation. All subsystems were implemented successfully: the logic for drone following, object detection and tracking, and construction of the net deployment mechanism all largely proceeded as our team expected from our Proposal. However, full end-to-end integration of the detection and tracking was not successful due to system-level hardware constraints (discussed further in Section 5.2).

4 Prototype

Our design prototype (Figure 5) implements real-time computer vision (CV) for object detection and tracking, drone controls logic for surveying and target shadowing, and a mechatronics assembly for accurate deployment of a mesh net for target enclosure.

Our prototype features an IMX219 monocular RGB camera fastened onto a RealSense T265 depth camera mounted at a 45° downward-oblique angle at the front end of the drone. Upon launching under Vicon positional control, the drone enters a “surveying” state, where it slowly rotates in place about its “home” position as it searches for a target. A trained



Figure 5: Final prototype drone.

CV model onboard in the IMX219 detects in real-time any suitable target within its field-of-view, and returns an appropriate 3D position of the target through correlation with the depth camera. The drone enters a “tracking” state, following the target from a fixed distance and ensuring it remains in the cameras’ line-of-sight. Upon request, the drone navigates overhead the target while it is stationary, after which the operator triggers the net deployment through a servo-actuated mechanism, before returning and landing at its home position.

Though the CV subsystem is operational with highly accurate real-time detection functionality, it was not fully integrated with the controls and net deployment subsystems (discussed further in Section 5.2). To demonstrate target tracking functionality without CV, a remote-control car equipped with a custom Vicon marker was used to provide the drone with a global-frame position of a moving target. This allowed the control and net deployment subsystems to be evaluated independently of the onboard detection pipeline.

4.1 Control Flow

All drone control logic is run via a ROS2 node on the onboard Jetson Nano. Information on the current position and orientation, target position and orientation, and urgent commands (such as landing) are communicated to the Orange Cube+ over MAVROS.

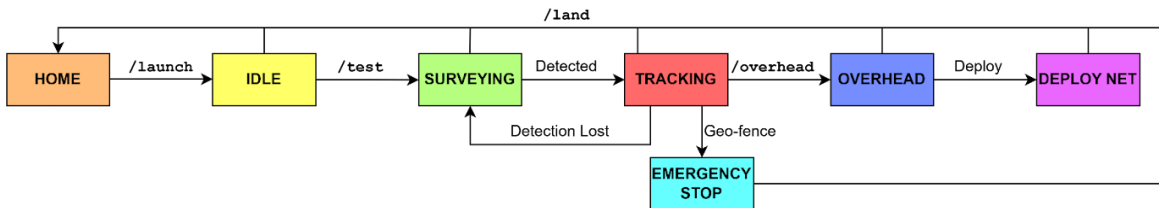


Figure 6: Final state machine architecture.

The overall state machine architecture of the prototype drone is presented in Figure 6. In “Home”, the drone is on the ground in its manually-placed position. Upon receiving a `/launch` ROS service command, the drone launches and maintains a stable hover 0.5 meters off the ground. Upon receiving a `/test` ROS service command, the drone enters “Surveying”, where it slowly rotates (i.e., incrementing its yaw) in discrete 30° increments about its current position – a 3-second yaw-hold duration was chosen to ensure the CV subsystem could reliably identify observed objects at each increment.

Upon reception of a valid “target pose” in the drone’s known global frame, the drone begins “Tracking” the target, maintaining a fixed relative position and always facing the target. If this “target pose” is ever lost for a sufficient duration of time, it re-enters “Surveying” from its current position – the implemented “target pose staleness” threshold is 2 seconds. While in “Tracking”, a human operator can then send an `/overhead` command when the target is determined by the operator as stationary. The drone will position itself above the last-known target pose under dead-reckoning (i.e., without CV feedback on the target location), specifically ensuring the center of the net deployment mechanism is atop the target. Upon actuation of a switch on the Taranis remote controller, the mechanism deploys the net. A `/land` command returns the drone to its home position on the ground.

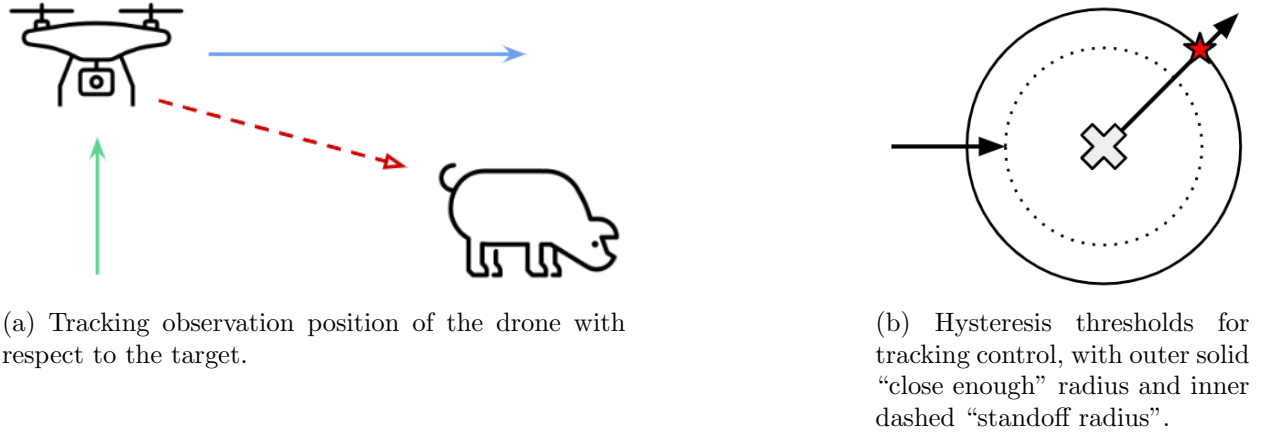


Figure 7: Drone control features.

Figure 7a presents the drone’s targeted hover position while “Tracking”, where the drone continuously adjusts its yaw to ensure it is precisely facing the target. The drone’s ideal position, from the current position upon receiving a “target pose”, is the “closest point” on the circle (centered at the target) of a desired “standoff radius” at the desired “hover altitude” of 0.5 meters. In practice, reliable identification of this ideal setpoint is challenging, as the drone is liable to overshoot this setpoint with improper PID tuning, resulting in a loss of tracking as the target escapes the drone’s vision, and erratic behaviour as the drone continuously identifies new “closest points” when overshooting to within the circle.

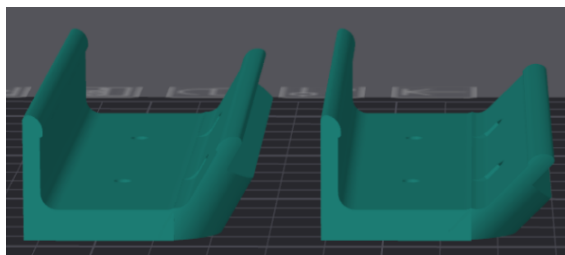
To simplify the control logic and prevent erratic, erroneous movements, the drone first targets the exact “target pose” (i.e., the centroid of the detected object at the “hover altitude”). Once within a “close enough” radius slightly larger than the “standoff radius”, the drone updates its setpoint to its exact current position – the radii are tuned to ensure that,

in practice, the drone’s expected overshoot results in a hover at the approximate “standoff radius”. As the target moves, the drone rotates to face it, but does not reposition itself until the target has moved beyond this “close enough” radius, at which point the drone repeats the procedure and eventually reaches a new suitable tracking pose. This hysteresis positional control logic is illustrated in Figure 7b, and results in significantly smoother and more stable control by minimizing the number and variety of unique positional setpoints while still ensuring the target never escapes the drone’s line-of-sight.

Regarding our objectives, our prototype can successfully track a target from a desired relative position (provided successful detection from other subsystems), prepare itself for accurate net deployment, and return home safely, all with limited human intervention. As discussed in Section 5.2, these objectives were achieved within the drone’s available flight-time, satisfying the “endurance” objective within the prototype’s scope.

4.2 Camera Mount

We developed a custom camera mount for our CV. We wanted our IMX219 camera to work with the RealSense T265 (and its ability to create a depth map) to determine the 3D pose of our detected target with respect to the drone. We had to ensure a grounded target was visible from a hovering drone and did not escape the cameras’ view as it moved, so positioning the cameras fully forward or downward was not viable. We tested various mounts angling the cameras 30° and 45° downward (from the frontal position) and found that 45° provided the best balance in terms of detection range as well as ensuring moving targets did not easily escape the drone’s line-of-sight.



(a) CADs of 30° and 45° mounts



(b) Final camera mount design on prototype.

Figure 8: Camera mount system.

However, the angled mount provided challenges in integrating with our existing chassis design. The foam landing pads below the drone obscured the bottom of the RealSense depth map, reducing its range and inducing positional drift (discussed further in Section 4.5) – this forced us to trim these foam pads to improve tracking performance.

4.3 Computer Vision

The CV pipeline begins with two primary sensing modalities: an RGB detection stream from the IMX219 camera and a stereo fisheye stream from the Intel RealSense T265. These

sensors serve complementary roles. The IMX219 provides high-resolution color imagery for object detection via a YOLOv8n neural network, while the T265 supplies both visual-inertial odometry and stereo imagery for geometric depth estimation. A third system, Vicon motion capture, is used externally for validation and ground truth but does not directly contribute to onboard CV.

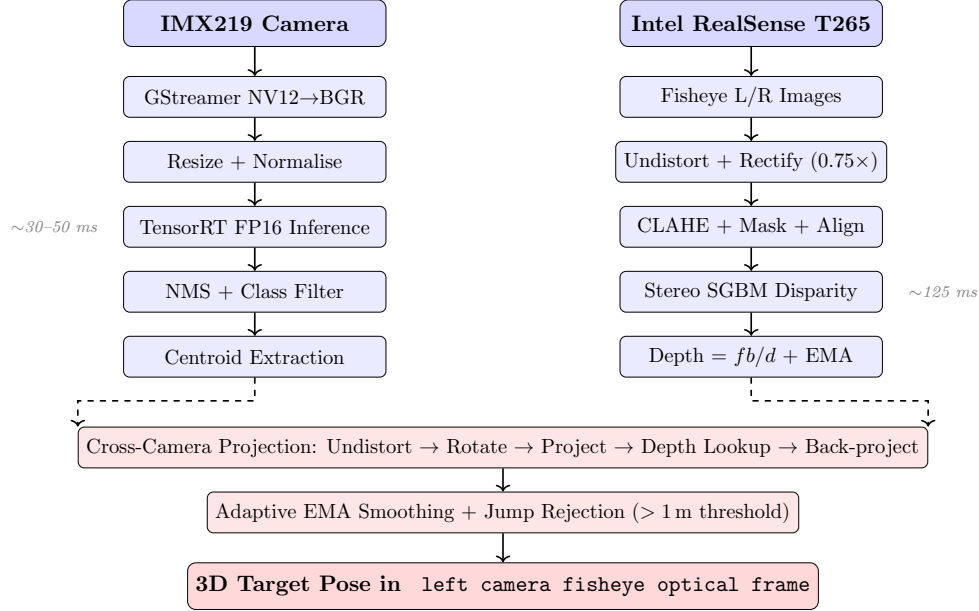


Figure 9: Complete end-to-end data flow from raw sensor data to 3D target localisation.

The RGB image processing pipeline works by integrating YOLOv8n, optimized for a single class. YOLOv8n is a data-driven detection model designed to run on edge devices such as the Jetson Nano. However, running its standard PyTorch variant is unrealistic on the Nano due to the sheer amount of throughput these models require. By building a TensorRT representation of our custom-trained YOLOv8n model, we were able to adequately conduct inference with minimal real-time latency. TensorRT itself is an SDK designed for high-performance deep learning inference on NVIDIA GPUs. It optimizes trained models from frameworks like PyTorch or ONNX, reducing latency and increasing throughput for production applications via techniques like quantization, layer fusion, and kernel tuning.

The data collection process itself involved acquiring 1000 IMX219 RGB frames of the target object and manually labeling them via bounding boxes. We collected images at different object orientations, drone altitudes, distances, and included distractive surrounding objects (i.e., chairs) to diversify our dataset. The lighting and general location of the object remained fixed in the flight-permitted area of Myhal 580. Training was conducted on an NVIDIA GeForce RTX 2080ti until 100 epochs elapsed, after which the loss was deemed low enough for use. After training on the curated dataset, the model was exported to ONNX format and compiled into a TensorRT engine using FP16 precision on the Jetson Nano.

Inference is executed asynchronously using PyCUDA within a ROS2 node to prevent the ROS2 executor from blocking and ensure MAVROS callbacks are serviced promptly.

Post-processing decodes the output into bounding boxes, filters them by confidence, and applies non-maximum suppression to remove duplicates. The centroid is then extracted and published to a dedicated centroid topic.

In parallel to object detection, a separate ROS2 node asynchronously computes depth using the T265 fisheye cameras. Distortion is modeled with a 4-parameter Kannala-Brandt model, and the resulting undistorted image is rectified to ensure horizontal scan lines during stereo matching. The undistorted and rectified image is then enhanced using histogram equalization to improve depth evaluation in low-contrast regions. Disparity itself is computed using a semi-global block matching technique that is well documented in literature, and the resulting disparity map is converted into depth using the fundamental stereo relation:

$$z = \frac{fb}{d} = \frac{(\text{focal length})(\text{baseline})}{\text{depth}}$$

To improve overall robustness and resistance to noise, techniques such as by-frame exponential moving average (EMA) and depth jump rejection were implemented to ensure a stable stream of data during vibrations and erratic drone maneuvers. The temporal filter is also dynamic. It adjusts its smoothing factor based on the drone’s velocity: when hovering, stronger smoothing reduces noise, while during rapid motion, the system prioritizes responsiveness by relying more heavily on the current frame.

The final component of the CV pipeline involves fusing 2D centroid detections with the depth produced by the fisheye cameras to produce a target 3D position that a control system may use for feedback. This process relied on precise intrinsic and extrinsic calibration. Intrinsic parameters model each camera’s projection characteristics, while extrinsic calibration – computed using Kalibr (third-party calibration tool), and an AprilGrid target – defines the rigid transformation between the IMX219 and T265 coordinate frames. First, the detected pixel value of the centroid is back-projected to produce its normalized image ray in the IMX frame using its intrinsics. Next, this ray is transformed into the left fisheye frame using the extrinsics acquired during calibration. Since the IMX was mounted between the fisheyes, the transformation was small but still aided in providing fine-grained accuracy.

By projecting this ray into the rectified depth image, our system identifies the corresponding pixel region from which depth is sampled. Rather than relying on a single pixel, we compute depth over a window whose size scales with the bounding box, improving robustness to noise and outliers. Once depth is obtained, we reconstruct the full 3D position by back-projecting along the ray. The resulting coordinates are expressed in the T265 optical frame and must be converted into a world frame for tracking.

As mentioned, all components of the CV system communicate through ROS2 nodes and topics. The architecture separates YOLOv8n inference, depth perception, and final object detection as distinct ROS2 nodes to create a modular system for overall 3D perception. Detection itself operates at approximately 20 Hz, depth estimation at 8 Hz, and fusion at the detection rate, producing a continuous stream of 3D target poses for controls to utilize.

In isolation, the CV pipeline works as intended. We achieved great pose accuracy (to within a few centimetres) with minimal noise and erratic behaviour. Integration errors are

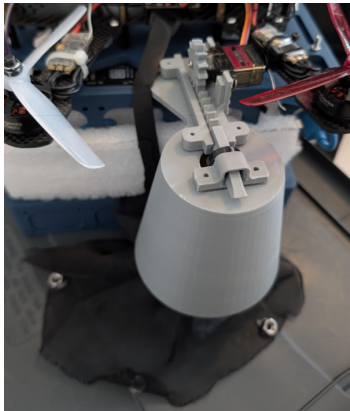
discussed in Section 5.2.

4.4 Net Deployment

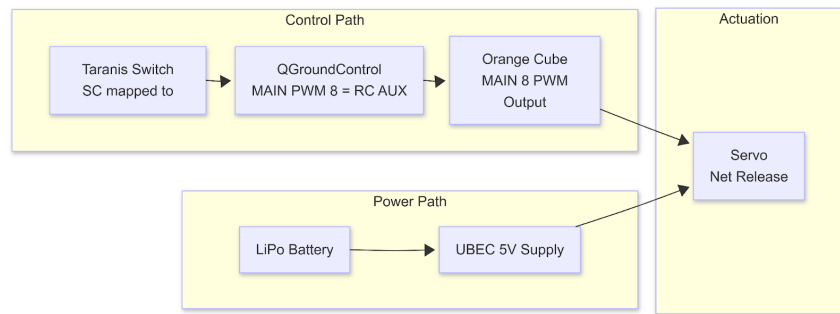
The net deployment functionality is carried out by a two-component system. The first component is the net, which will enclose the pig, and the second component is the mechatronic device, which will actuate the deployment of the net. The design consists of a bolt latch with a rack and pinion that holds the center of the net via a connected hook looped around the rack. The net hangs down from the hook and is protected from external hardware by a downward-facing cup that encloses the slack net. The net is dropped by translating the rack via the servo.

The net deployment subsystem (Figure 10a) is actuated through the following routine:

1. Net is manually loaded into the device by hooking the center loop about the bolt rack.
2. Drone initiates flight sequence and navigates above target.
3. User actuates the appropriate switch on the Taranis controller to signal the antenna on the drone.
4. Signal is passed from the Cube to the servo, which turns $\approx 180^\circ$.
5. The connected pinion gear rotates along with the servo and translates the rack mounted underneath.
6. As the rack retreats, the net's center loop falls from its attachment and the net is effectively deployed from the drone.



(a) Final net deployment design on prototype.



(b) Net deployment control and power architecture. RC input is mapped to MAIN PWM 8 on the Cube Orange+ to actuate the servo, while a UBEC provides stable power to prevent interference with onboard systems.

Figure 10: Net deployment system.

The net deployment subsystem required a reliable method of actuating a servo onboard the drone during flight. Three key design considerations shaped this design: compatibility

with the existing flight-control architecture, reliability of operator actuation, and electrical robustness under the servo’s transient current draw.

We first explored controlling the servo programmatically through PX4 using MAVROS commands, which would have allowed deployment to be triggered directly through the on-board software stack. However, this approach did not provide reliable behaviour in practice, due in part to firmware and actuator-mapping limitations encountered during integration. A second option was to use a separate microcontroller to actuate the servo independently, but this would have introduced additional hardware and increased system complexity for a subsystem whose function was fundamentally discrete and operator-triggered.

The final implementation used the Cube Orange+ directly to generate the servo command. The output was configured through the actuator mapping in QGroundControl and linked to a dedicated switch on the Taranis transmitter. This allowed the operator to trigger release manually at the appropriate moment. To ensure stable operation, the servo was powered through a UBEC rather than from the Jetson Nano, preventing current spikes when actuating the servo from affecting other onboard electronics. Overall, this approach provided a simple, reliable, and well-integrated solution for prototype-level deployment. The control and power architecture of this system is shown in Figure 10b.

4.5 Safety Features

Our drone operated entirely under Vicon control; its noise-free global position through a Vicon marker was continuously provided to MAVROS. Our drone successfully navigated under RealSense position control in previous Flight Exercises. However, in our testing, we encountered various instances of spontaneous RealSense drift only occurring during flight (presumably from vibrations), causing catastrophic crashes. Prioritizing productive design work rather than the rebuild time that might be required in the event of such crashes, we chose to use Vicon with the confidence we could get RealSense control working with minimal effort in future iterations; since RealSense control was achievable in the Flight Exercises, we did not see it as a necessary element of our minimum viable product given our time constraints.

However, we did implement and validate a RealSense control option with a Vicon “fallback” safety net. Under this control regime, both RealSense and Vicon data were received, though only RealSense data would be provided to MAVROS. On each received reading, the position and orientation information between both sources was compared – if the RealSense data diverged sufficiently, the source became invalidated, and all MAVROS control shifted to Vicon for the remainder of the flight. In practice, the bounds for divergence (0.3 meters positionally, 0.1 radians orientationally) were tuned empirically to ensure that expected noise from RealSense data would not trigger invalidations, but that trends in positional drift were caught before the drone became unstable. In particular, this control regime allowed identification of gradual RealSense drift (breaching the positional divergence boundary after ≈ 10 s at a stable hover) caused by the obscuring of the RealSense’s view by our prototype’s styrofoam landing pads. Without this fallback mechanism, this RealSense drift could have caused a catastrophic failure requiring rebuilding and wasting valuable testing time.

Various measures also ensured our drone was unable to travel too far in the event of any divergent lateral control. An emergency stop mechanism was implemented via a “geofence” feature: if the drone ever traveled too far laterally (i.e., 3 meters) from its home position, it would enter a holding state from which only `/land` and `/abort` commands would be processed. As well, received tracking targets beyond a certain distance (i.e., 2 meters) were immediately rejected, to ensure targets produced by erroneous CV results would not compel the drone to fly to an implausible, unsafe location. Finally, the lateral velocity limits on the Cube Orange+ were heavily reduced to 1 m/s to allow for safer manual takeover in the event of control failures.

Though not explicitly included in our team’s objectives, these key measures ensured our drone operated as safely as possible to prevent crashes and allow us sufficient time to achieve all design objectives (discussed further in 6.2).

4.6 Scaling Up

As our prototype successfully demonstrates the functionality of all necessary subsystems using relatively low-cost commercial-off-the-shelf hardware, scaling up the design for real-world implementation is a straightforward task. Hardware selection of a larger, weather-proof drone with a longer-lasting battery, stronger propulsion systems, and more deft controls would allow for satisfaction of our “endurance” objective. Collection of a more robust training dataset and implementation of numerous higher-quality RGB and depth cameras (or perhaps other suitable vision systems like 3D or thermal cameras [9]) would allow for straightforward scaling of our detection systems to suitable levels. Expanding our “Surveying” algorithm to a proper area search (i.e., navigating the entire region rather than just rotating in-place) is also a straightforward, proven control task. Though our prototype hardware demonstrated successful depth camera-based control during previous flight exercises (which may be used analogously in real-world implementations), integration of an accurate, real-time GPS module is reasonable at scale and extends logically from our Vicon control regime. Finally, as full-size catch-and-release nets are already fairly lightweight, the direct expansion of our current mechanism is feasible at scale.

5 Results

All prototype systems were independently verified to ensure isolated functionality. The integrated prototype was then validated in a suitable end-to-end test, mocking a real-world design use-case in the prototyping space available in Myhal.

5.1 Verification

5.1.1 Controls

A remote-control car equipped with a custom Vicon marker was used to provide the controller a noise-free global-frame “target pose” (Figure 11). The equivalent “target pose”

provided when computer vision (CV) is used for tracking is instead provided in the camera frame (i.e., relative to the drone’s current position). This Vicon proxy allows for verification of all control logic in tandem with CV development: the only integration burden for these subsystems is the conversion of the camera frame to the global frame (which was verified easily with a table-top test), rather than the entire end-to-end controls pipeline.



Figure 11: Remote-control car equipped with Vicon marker and yellow plushie used in CV training.

As the control logic built upon Flight Exercise 3 and the state machine architecture permitted easy addition of new behaviours without adjustment to earlier behaviours, verification of basic control features (i.e., holding a hover, aborting, tracking a stationary setpoint, including rotating in-place) was not needed at this prototyping stage. As discussed in Section 4.1, tuning the controller to track a target from a fixed position proved challenging and required implementation of careful hysteresis thresholds. In particular, the drone consistently overshoot provided setpoints by large margins (particularly if they required a change in yaw). Qualitatively, it was clear that the overshoot behaviour of initial implementations would cause the target to escape the camera’s line-of-sight, which was a specific concern outlined in our Proposal. As adjustment of the flight controller’s internal PID values was time-consuming and challenging to interpret intuitively, software solutions (such as the tuning of hysteresis radius bounds or adjustments to setpoint update rates) were instead used to achieve successful tracking behaviours.

This subsystem’s performance against our objectives of autonomous tracking and safely returning home was primarily verified qualitatively. The drone appeared to successfully keep the target in sight while flying stably around the arena, and the exact parameters of hysteresis radii could be easily tuned upon integration of the CV system if they proved unsuitable for maintaining a constant line-of-sight. Similarly, the logic to navigate the drone overhead the target appeared successful, but the exact positioning could not be verified until integration with the net deployment system (to observe where the net dropped).

5.1.2 Computer Vision

CV performance was verified under three key criteria: accuracy, noise, and latency. For real-time detection, accuracy is paramount. Several trials of desk-mounted CV trials were conducted to monitor the pose output as our boar model was maneuvered. Object detection showed great generalizability in several lighting conditions as well as in shadows. Attaching a

custom Vicon marker to the model did not compromise consistent and accurate detections, enabling Vicon fallback in case CV diverges from ground truth. Latency and noise were more burdensome issues. To decrease depth computation latency, we opted to compute a sparse disparity map about points near an isolated centroid. To smooth noise, adaptive EMA was employed, and depth was conditionally computed if the confidence of the bounding box detection exceeded 70%. The final result showed minimal noise ($\pm 2\text{cm}$) in static drone conditions and could run with minimal latency in the absence of MAVROS overhead. Both YOLOv8n and depth smoothing techniques offered superior perception under vibrational load; during flight, real-time object detection was not hindered.

5.1.3 Net Deployment

Potential designs to satisfy the requirements of the net subsystem were developed through an iterative process.

Our first design consisted of four servo-actuated grippers at each corner of the bottom of the drone chassis. The net is held tautly by the four grippers such that there is no slack during drone flight, and it is dropped by actuation of the servos. This design concept's issue was that the space at the bottom of the drone chassis is very congested with hardware equipment and protective padding. This complicates the implementation of the net deployment device in this area. Furthermore, the usage of four servos is not optimal in terms of minimizing the power consumption of the device. An improved design would be able to be affixed to only the available space on the side of the drone and use a minimal number of servos.

Our second design consisted of one claw gripper, which holds the center of the net. The net hangs down from the gripper and is protected from external hardware by a downward-facing cup that encloses the slack net. The net is dropped by actuating the gripper via the servo. This concept is optimal in terms of its usage of free space as well as the minimal number of servos required. An issue that presented itself, however, was that the reliability of the claw gripper design is degraded by motion and vibrations during flight. Thus, an improved design would grip the net more securely, such that it performs robustly against vibrations and perturbations experienced during flight.

Our final design consisted of a bolt latch with a rack and pinion, which holds the center of the net via a connected hook looped around the rack. The net hangs down from the hook and is protected from external hardware by a downward-facing cup that encloses the slack net. The net is dropped by translating the rack via the servo. This concept is optimal in terms of its usage of free space as well as the minimal number of servos required. It further improves upon the previous concept's shortcomings by securing the net reliably during flight.

The net deployment function of the final design was verified in terms of capture accuracy across trials. 40 net deployment trials were conducted, obtaining a $\approx 95\%$ success rate. For each trial, net deployment was commanded via the RC transmitter with the drone posed above the stationary pig at the typical tracking altitude. The reliability of the deployment system was supported by the control architecture in Figure 10b, with servo actuation handled directly by the Orange Cube and powered through a regulated supply. This eliminated issues with inconsistent triggering and prevented electrical disturbances during operations.

5.2 Validation

The prototype was validated through a mock test in the Myhal prototyping space, simulating an end-user use-case. For our validation routine:

1. The drone is issued `/launch` and `/test` command to begin surveying.
2. The drone identifies the RC car through CV, and begins tracking it.
3. The drone follows the target, and demonstrates the ability to re-enter survey and re-detect the target upon a line-of-sight breach.
4. Upon request, the drone moves overhead the stationary target.
5. The net is deployed and lands atop the target.
6. The drone returns home and lands.

Though the “accurate identification” objective could not be expressly validated, all other objectives – “tracking”, “net deployment”, and “safe return” – were validated in one continuous flight (thus meeting the “endurance” requirement). Our final validation routine did not include CV, instead tracking the target via publishing Vicon information for the car – the “line-of-sight” loss and re-detection was simulated via ceasing publication of this Vicon data, then restoring it. Though the net was observed qualitatively to drop accurately atop the target, the downwash from the propellers while hovering so near the ground immediately pushed the net away upon landing. As our prototype intentionally flies at a lower altitude with a smaller, lighter net than our intended final design, this issue was symptomatic of prototype constraints rather than any deeper validation-level failure of the design concept.

The drone was observed to fly erratically when the CV model ran during flight; the setpoints provided over MAVROS were completely ignored, and the drone could not maintain a stable hover. In particular, it appeared the drone was flying well above the setpoint height of 0.5 meters provided in the “Idle” state, and oscillated wildly from approximately 0.7 to 1.5 meters off the ground. Even when CV inference was not used as a control source (i.e., the “target pose” was provided with Vicon data), the drone was entirely unable to track it; the drone appeared to reject any provided setpoint and continue to hover unstably.

These behaviours indicate MAVROS communication failures arising from the operation of all subsystems, perhaps due to power usage or scheduling issues caused by the CV subsystem’s implementation of multi-threading. However, as ROS is intended to function as a real-time operating system, and our drone exhibited numerous power-related issues throughout the entire course (discussed further in [6.1](#)), the root issue is suspected to be hardware- and power-related.

As such, even though our drone could not technically complete all validation tasks, we are confident in the performance of all subsystems and the ability to integrate them all with more appropriate hardware: we have successfully demonstrated that we built “the right solution” to our design problem.

6 Lessons Learned

6.1 Power and Flight-Time Constraints

A key limitation encountered throughout the project was limited battery life and overall system power constraints. Even during partial operation, such as standalone tracking, the platform achieved no more than 3-4 minutes of flight-time, significantly limiting our iteration and testing cycles. While both the computer vision and control subsystems were validated independently, these limitations became more evident during system integration.

When multiple subsystems were run simultaneously, the system exhibited instability. MAVROS communication became less reliable, and the drone was unable to maintain stable flight under full onboard processes. Through testing, this behavior was determined not to be caused by issues in the individual algorithms, but rather by system-level resource limitations.

Under full load, the platform showed reduced flight-time and signs of elevated power draw, including battery stress during operation. One possible contributing factor is increased current draw from the propulsion system. Earlier in the project, differences in individual motor sounds were observed, but they did not clearly indicate a critical issue at the time. In hindsight, these may have been early indicators of underlying system limitations, although the exact root cause could not be conclusively isolated within the available development timeframe.

Overall, these findings indicate that the primary limitation of the system was not in perception or control capability, but in available power and system resources. This limitation also prevented full utilization of scheduled testing sessions, as the drone could fly for only a fraction of our allotted time, and the time required to replace the battery and reinitialize the system far exceeded the remainder of our testing window. With improved power capacity or further investigation into propulsion and load distribution, stable real-time integration of all subsystems is expected to be achievable.

From a design process perspective, we did not initially treat flight-time as a critical system constraint within our control, instead attributing it primarily to battery condition. A key lesson from this experience is the importance of validating system performance through direct measurement rather than relying on expected or assumed hardware behavior. Early in the project, short flight times were attributed to battery condition and were not treated as a critical system constraint. It was only after observing battery degradation and testing with replacement batteries (with empirically known flight-time on other drones) that the persistence of limited flight-time indicated a deeper system-level issue. In future iterations, we would prioritize early flight-time benchmarking and treat deviations from expected performance as signals requiring further investigation. For future teams, we recommend verifying key system constraints through direct testing, rather than relying solely on nominal specifications or prior expectations.

6.2 Crash Safety

Our drone flew entirely under Vicon control and never operated under RealSense control without an available Vicon fallback mechanism. This decision was made after encountering various RealSense drift issues during Flight Exercise 2, which were irreproducible outside of flight scenarios but caused catastrophic crashes. These crashes were demoralizing and time-consuming, bottlenecking meaningful design work without allowing the opportunity for productive debugging of the issue. To prevent such frustrating iteration loops from reoccurring, this “safety net” of absolute Vicon positional data was used, and allowed for successful identification of further RealSense drift issues arising from our novel camera configuration.

The primary lesson we learned in this process was to prioritize the safety of the drone above everything else. Drones are extremely fragile, dangerous, and unforgiving devices. Given the limited development and active testing opportunities available, measures should be taken to ensure that crashes are rendered nearly impossible by the control logic.

6.3 Team Task Management

A key challenge in this project was structuring the work such that all team members could make meaningful technical contributions in parallel. Early in the design process, we recognized that much of the project scope, particularly in control and computer vision, was highly centralized and difficult to divide into independent tasks. These subsystems were tightly coupled and relied on shared drone hardware for testing, which introduced bottlenecks and limited opportunities for concurrent development.

To address this, we made a deliberate design decision to incorporate an additional mechatronic subsystem that could be developed, tested, and validated with minimal dependence on the full system. This allowed team members to work more independently while still contributing to the overall project, improving both productivity and engagement. This experience showed us that designing for task independence early on is an effective strategy for enabling parallel development and ensuring meaningful team contribution. For future teams, we recommend considering task independence as a design objective early on to enable more effective parallel development.

7 Conclusion

Overall, our prototype demonstrated the feasibility of autonomous target detection, tracking, and assisted capture using a net deployment mechanism. Across the project, the core subsystems for computer vision, control, and net deployment were each developed and validated independently, showing that the proposed system architecture is technically viable. While full end-to-end integration was limited by hardware and power constraints, these challenges revealed important system-level considerations for future development. Taken together, the results support the value of the overall design concept as a potential tool for invasive wild pig monitoring and capture, and identify clear directions for improving real-world implementation.

References

- [1] College of Agriculture and Bioresources, “Wild pigs invade canadian provinces—an emerging crisis for agriculture and the environment,” University of Saskatchewan, 2019, [Online; accessed 11-Feb-2026]. [Online]. Available: <https://agbio.usask.ca/news/2019/05/wild-pigs-invade-canadian-provincesan-emerging-crisis-for-agriculture-and-the-environment.php>
- [2] R. A. Aschim and R. K. Brook, “Evaluating cost-effective methods for rapid and repeatable national scale detection and mapping of invasive species spread,” *Scientific Reports*, vol. 9, no. 1, May 2019.
- [3] A. Bamford, ““A Very Serious Crisis”: Canada’s Wild Pig Population Poses Threat to Neighbouring U.S. States,” CTV News, 2024, [Online; accessed 12-Feb-2026]. [Online]. Available: <https://www.ctvnews.ca/canada/article/a-very-serious-crisis-canadas-wild-pig-population-poses-threat-to-neighbouring-us-states/>
- [4] Animal Health Canada, “Canada’s invasive wild pig strategy: 2022–2032,” Animal Health Canada, Tech. Rep., 2022, [Online; accessed 11-Feb-2026]. [Online]. Available: <https://animalhealthcanada.ca/canadian-invasive-wild-pig-report>
- [5] Animal and Plant Health Inspection Service, “Identifying feral swine,” USDA APHIS, 2025, [Online; accessed 12-Feb-2026]. [Online]. Available: <https://www.aphis.usda.gov/operational-wildlife-activities/feral-swine/identification>
- [6] J. W. Fischer, L. J. Miller, T. Guerrant, K. C. VerCauteren, and K. M. Pepin, “Scouting with drones to increase efficiency of locating wild pigs for removal,” *Wildlife Society Bulletin*, vol. 49, no. 4, Nov. 2025.
- [7] Government of Ontario, “Ontario’s strategy to address the threat of invasive wild pigs,” Government of Ontario, Tech. Rep., 2021, [Online; accessed 11-Feb-2026]. [Online]. Available: <https://ero.ontario.ca/public/2021-04/Wild%20Pig%20Management%20Plan%20E%20FINAL.pdf>
- [8] Ministry of Natural Resources, “Annual report on invasive wild pig sightings: 2023–2024,” Government of Ontario, Tech. Rep., 2024, [Online; accessed 11-Feb-2026]. [Online]. Available: <https://www.ontario.ca/files/2024-12/mnr-invasive-wild-pigs-in-ontario-annual-report-en-2024-12-03.pdf>
- [9] M. Kim, O.-S. Chung, and J.-K. Lee, “A manual for monitoring wild boars (*sus scrofa*) using thermal infrared cameras mounted on an unmanned aerial vehicle (uav),” *Remote Sensing*, vol. 13, no. 20, p. 4141, Oct. 2021.
- [10] National Instruments, “Simple state machine template documentation,” Feb. 2025, [Online; accessed Apr. 10, 2026]. [Online]. Available: <https://www.ni.com/en/support/documentation/supplemental/16/simple-state-machine-template-documentation.html>